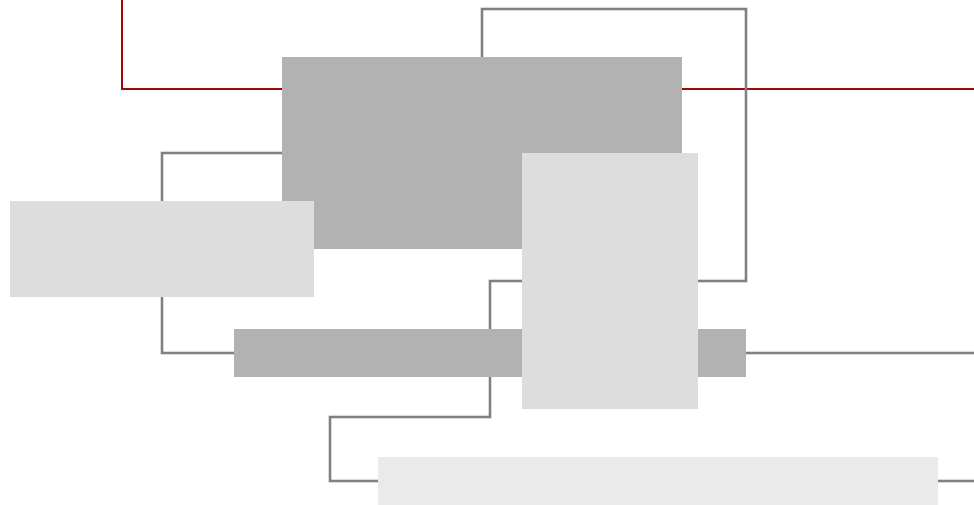


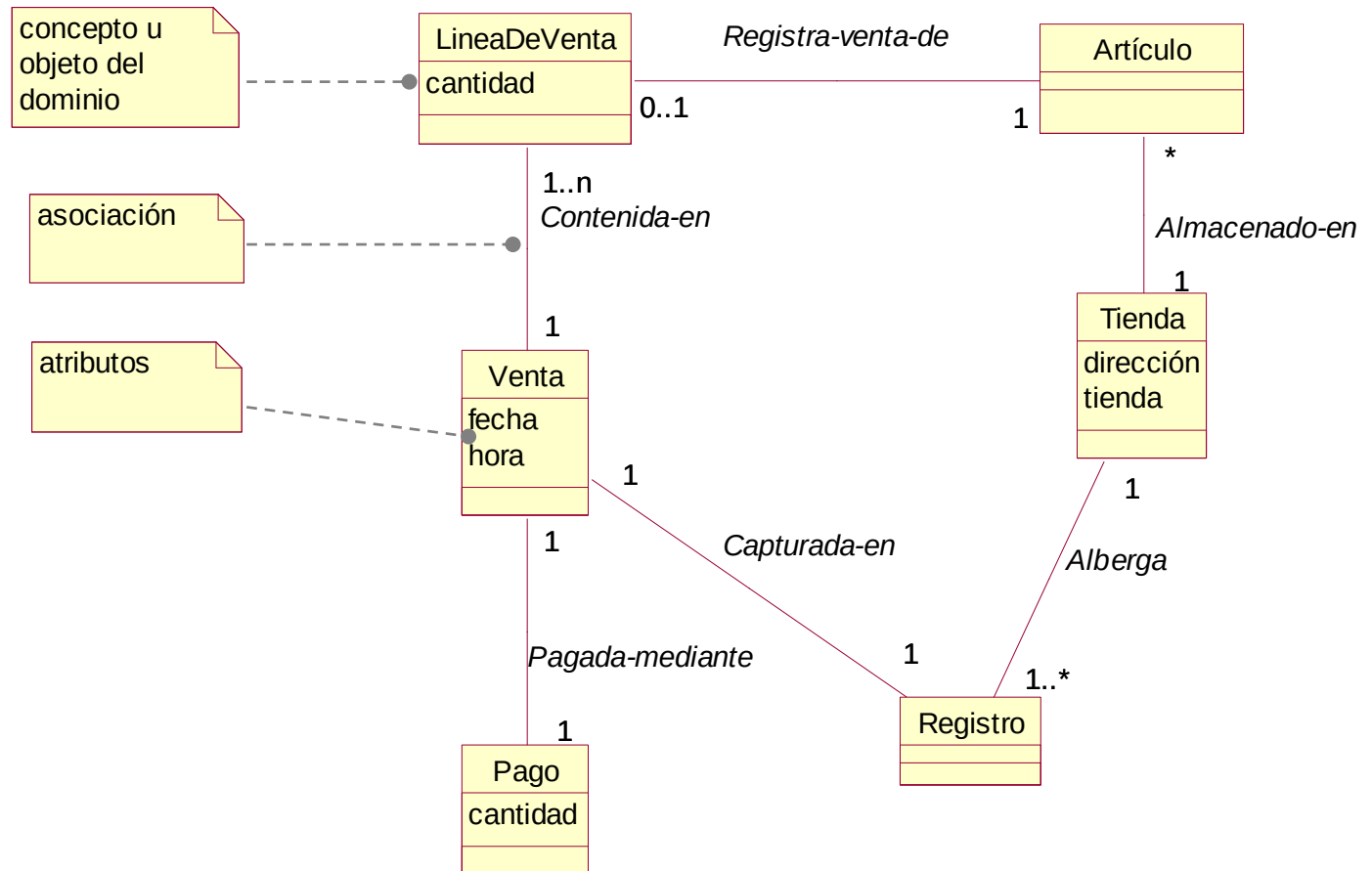
Modelo de Dominio



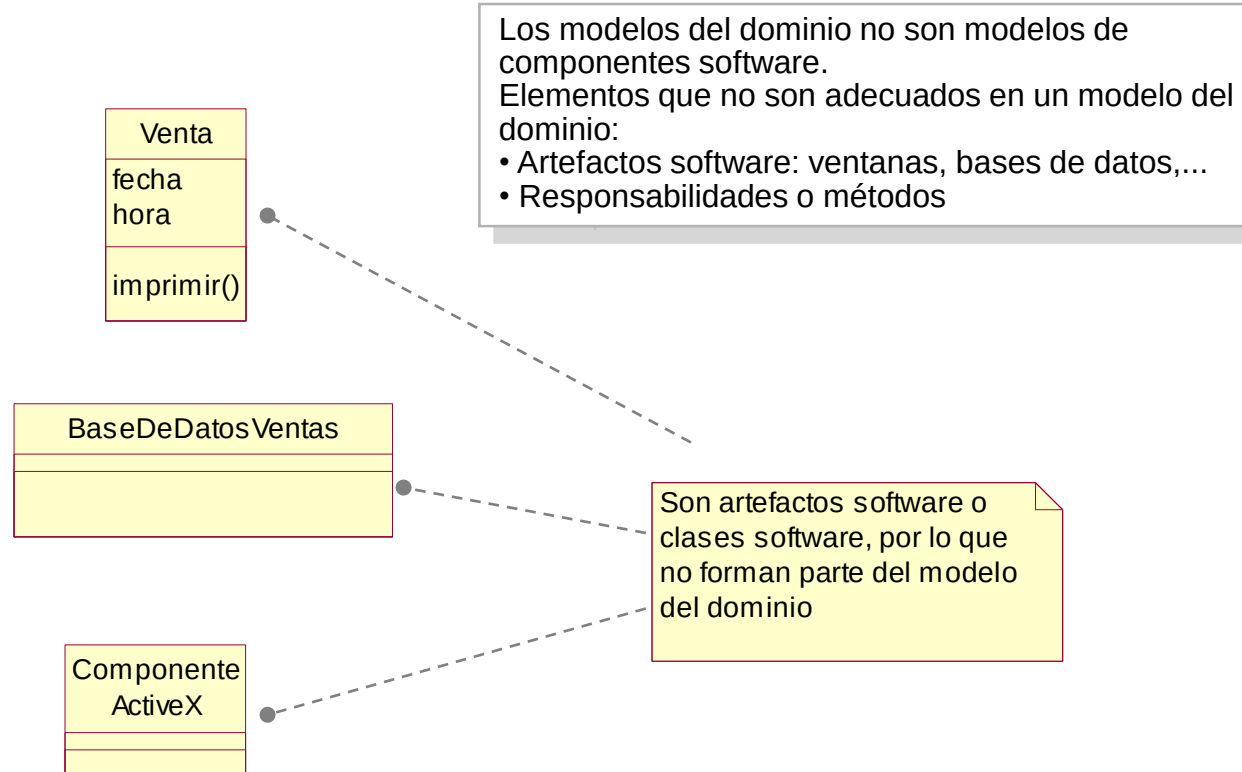
- Artefacto de UML: modelo del dominio (o modelo conceptual)
 - muestra clases conceptuales significativas en un dominio del problema, es decir, en el mundo real
 - no muestra componentes software, clases software u objetos software con responsabilidades
 - es el artefacto más importante en un análisis orientado a objetos (AOO)
 - UML utiliza **diagramas de clases** para representar el modelo del dominio, que muestran:
 - objetos del dominio o clases conceptuales
 - asociaciones entre las clases conceptuales
 - atributos de las clases conceptuales
- **principal tarea del AOO:** identificar diferentes conceptos en el dominio del problema y documentar el resultado en un modelo del dominio
- ejemplo: en el dominio de ventas pueden identificarse clases conceptuales como *Tienda*, *Registro* o *Venta*.

modelo del dominio:definición

Modelo del dominio: ejemplo de un Diagrama de Clases



modelo del dominio: definición



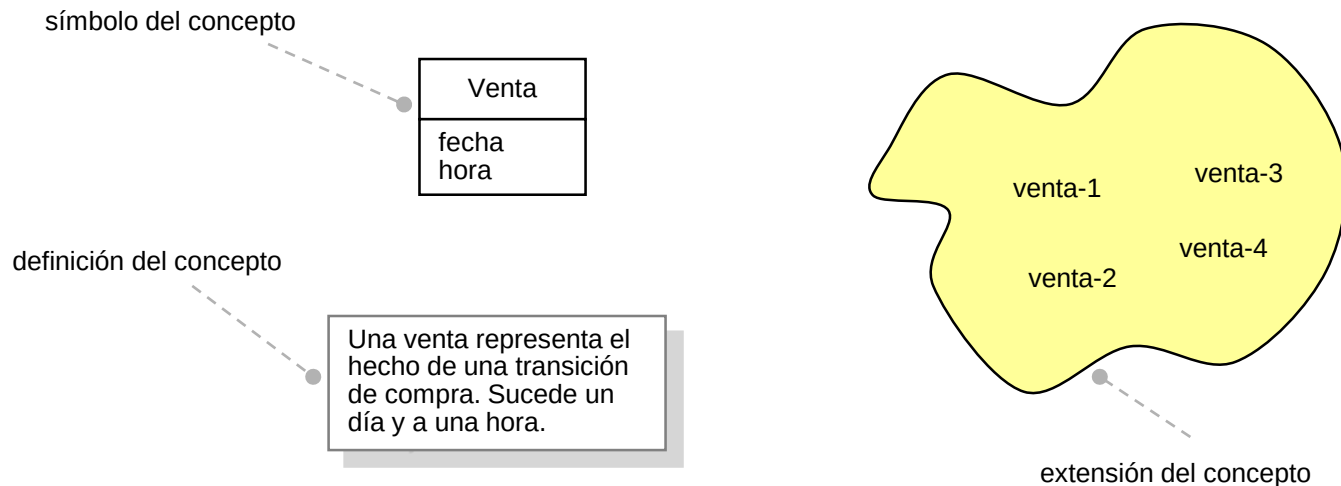
- pasos en la realización de un modelo del dominio
 1. identificar y listar las clases conceptuales candidatas
 2. representarlas en un modelo del dominio
 3. añadir las asociaciones necesarias para registrar las relaciones que hay que mantener en memoria
 4. añadir los atributos necesarios para satisfacer los requisitos de información

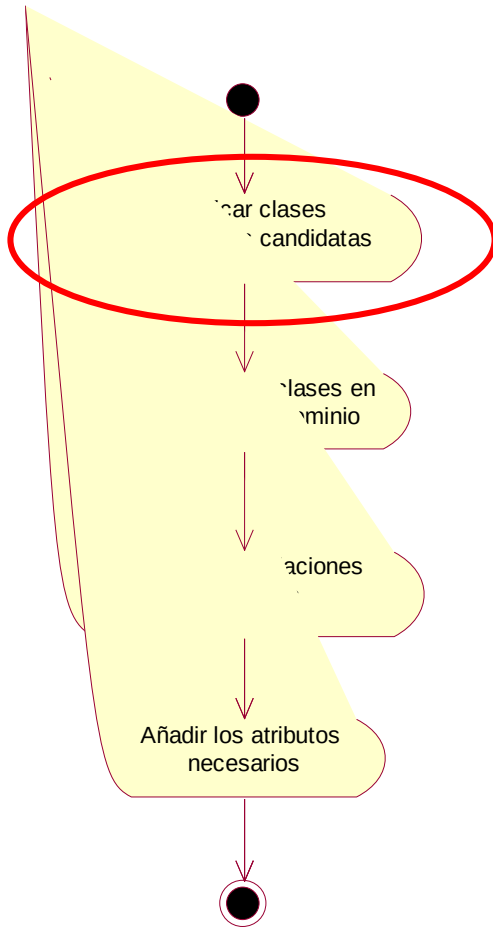
- importante:

- utilizar el vocabulario del dominio al nombrar las clases conceptuales y atributos
- excluir las características irrelevantes
- no añadir cosas que no se encuentran en el dominio del problema que se está estudiando

■ clases conceptuales

- informalmente: una clase conceptual es una idea, cosa u objeto
- formalmente puede considerarse en términos de:
 - símbolo: palabras o imágenes que representan una clase conceptual
 - definición de la clase
 - extensión: conjunto de ejemplos a los que se aplica la clase





- identificación de clases conceptuales
 - para cada escenario se identifican las clases conceptuales relacionadas con él
 - mejor especificar en exceso un modelo del dominio con muchas clases conceptuales "de grano fino" que especificar por defecto
 - estrategias complementarias para identificar clases conceptuales
 - utilización de una lista de categorías de clases conceptuales
 - identificación de frases nominales

modelo del dominio: identificación de clases

- identificación de clases conceptuales mediante una lista de categorías
 - se utiliza una lista de categorías habituales que normalmente merece la pena tener en cuenta

Categoría de clase conceptual	Ejemplos
objetos tangibles o físicos	<i>Registro</i> <i>Avión</i>
especificaciones, diseños o descripciones de las cosas	<i>EspecificacionDelProducto</i> <i>DescripcionDelVuelo</i>
lugares	<i>Tienda</i>
transacciones	<i>Venta, Pago</i> <i>Reserva</i>
líneas de la transacción	<i>LineaDeVenta</i>
roles de la gente	<i>Cajero</i> <i>Piloto</i>
contenedores de otras cosas	<i>Tienda, Almacén</i> <i>Avión</i>
cosas en un contenedor	<i>Artículo</i> <i>Pasajero</i>
otros sistemas informáticos o electromecánicos externos al sistema	<i>SistemaAutorizaciónPagoCrédito</i> <i>ControlDeTraficoAereo</i>

- análisis del lenguaje natural: identificación de clases conceptuales mediante frases nominales
 - heurística de Abbot (1983)
 - identificar nombres y frases nominales en las descripciones textuales de un dominio, considerándolos como clases conceptuales o atributos candidatos
 - punto débil: imprecisión del lenguaje natural, ambigüedades (sinónimos, redundancias,...)
 - se realiza a partir de las descripciones completas del dominio del problema

Escenario principal de éxito (o flujo básico) de Procesar Venta.

1. El **Cliente** llega al **terminal PDV** con **mercancías y/o servicios** que comprar
2. El **Cajero** inicia una nueva **venta**
3. El Cajero introduce el **identificador del artículo**
4. El Sistema registra la **línea de venta** y presenta la **descripción del artículo, precio y suma** parcial. El precio se calcula a partir de un conjunto de reglas de precios.
El Cajero repite los pasos 3-4 hasta que se indique
5. El Sistema muestra el total con los **impuestos** calculados
6. El Cajero le dice al Cliente el total, y solicita el **pago**
7. El Cliente paga y el Sistema gestiona el pago
8. El Sistema registra la venta completa y envía la información de la venta y el pago al sistema de **Contabilidad** externo (para la contabilidad y las **comisiones**) y al sistema de **Inventario** (para actualizar el inventario).
9. El sistema presenta el **recibo**
10. El Cliente se va con el recibo y las mercancías

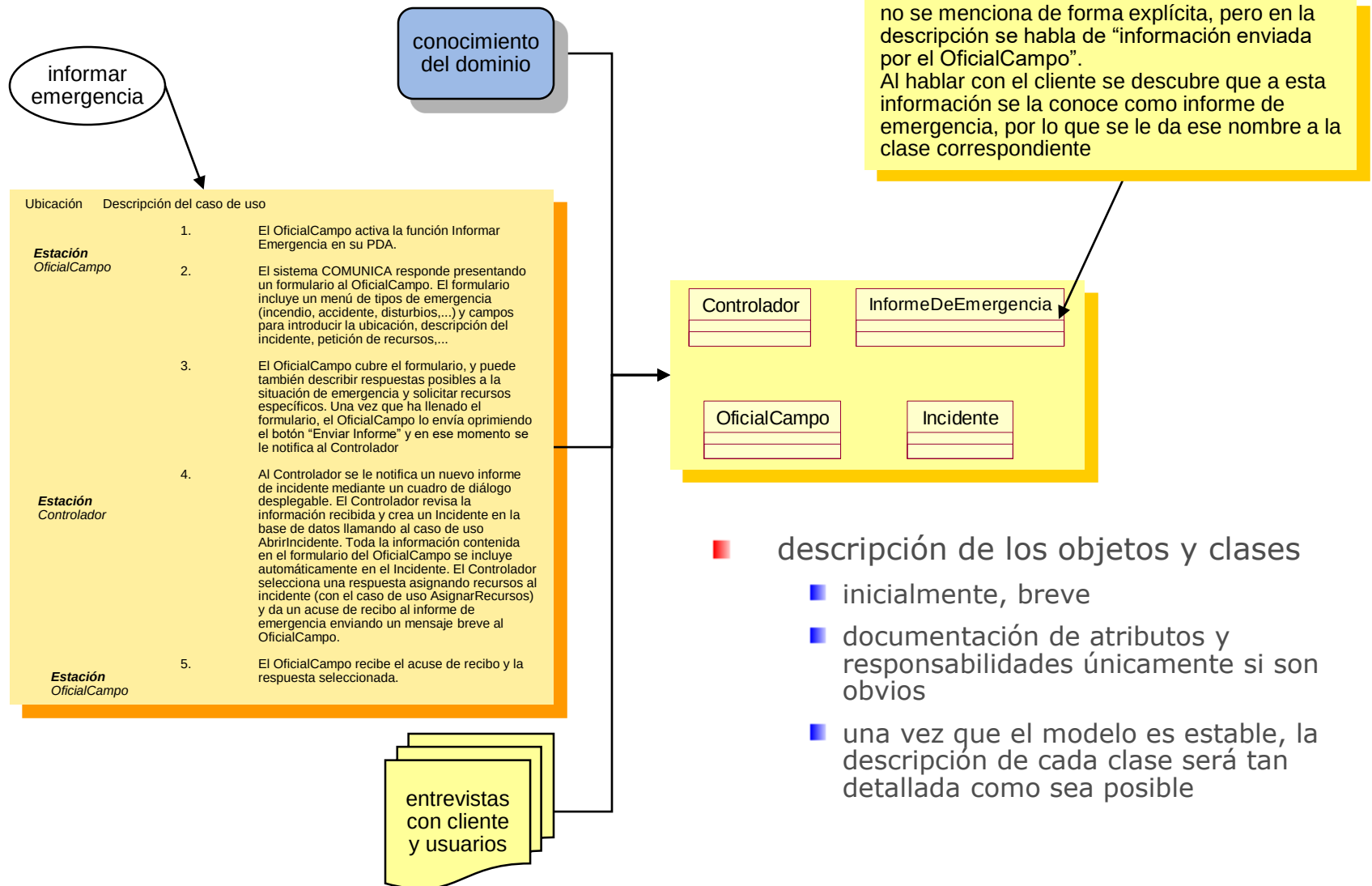
Extensiones (o flujos alternativos)

...

7a. Pago en efectivo:

1. El Cajero introduce la **cantidad** de dinero **entregada** en efectivo.
2. El sistema muestra la **cantidad** de dinero **a devolver** y abre el **cajón de caja**.
3. El Cajero deposita el dinero entregado y devuelve el cambio al Cliente
4. El Sistema registra el pago en efectivo

modelo del dominio: identificación de clases



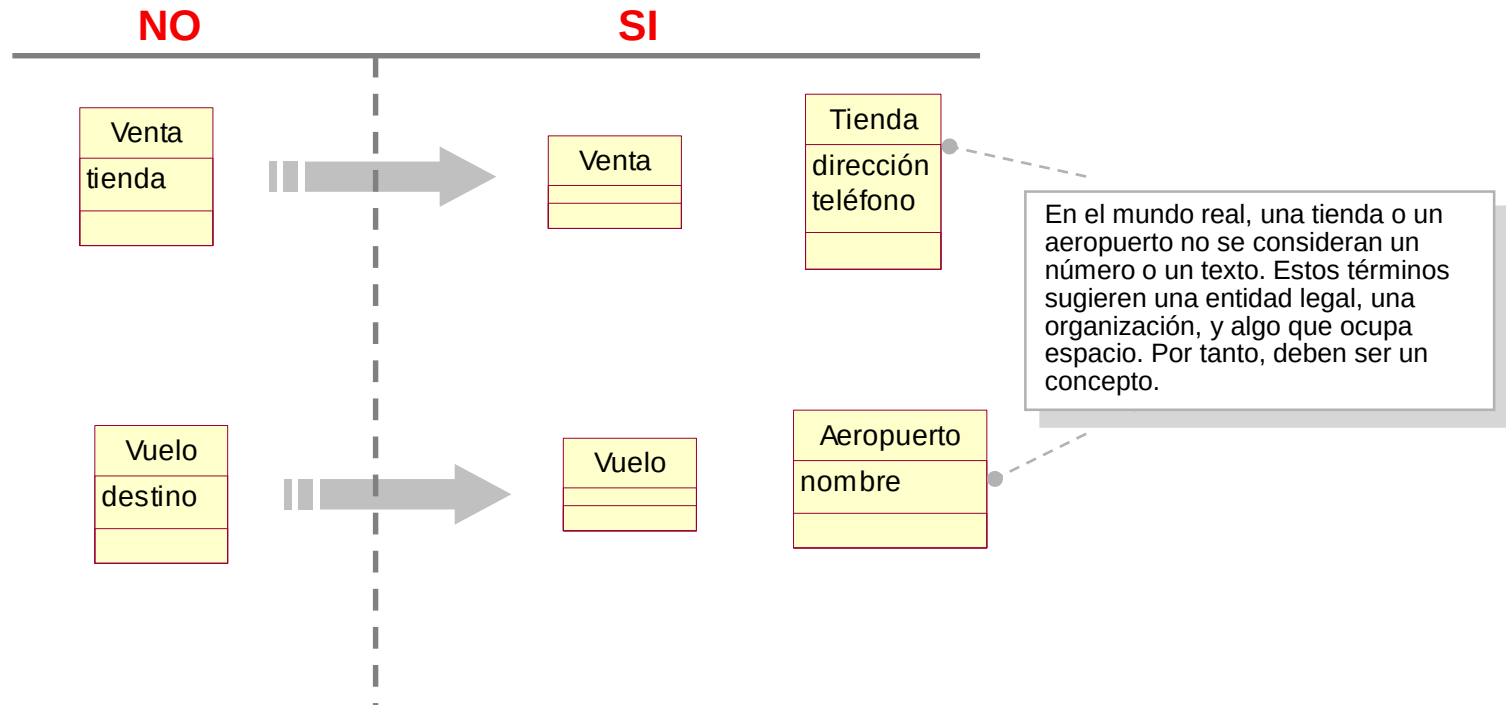
- lista de clases conceptuales candidatas del dominio
 - generada a partir de
 - lista de categorías
 - análisis de lenguaje natural (frases nominales)
 - restringida a los requisitos que se están estudiando actualmente
 - ejemplo: lista de clases candidatas del caso de uso *Procesar Venta*.

Registro	EspecificaciónDelProducto
Artículo	LíneadeVenta
Tienda	Cajero
Venta	Cliente
Pago	Encargado
CatálogoDeProductos	...

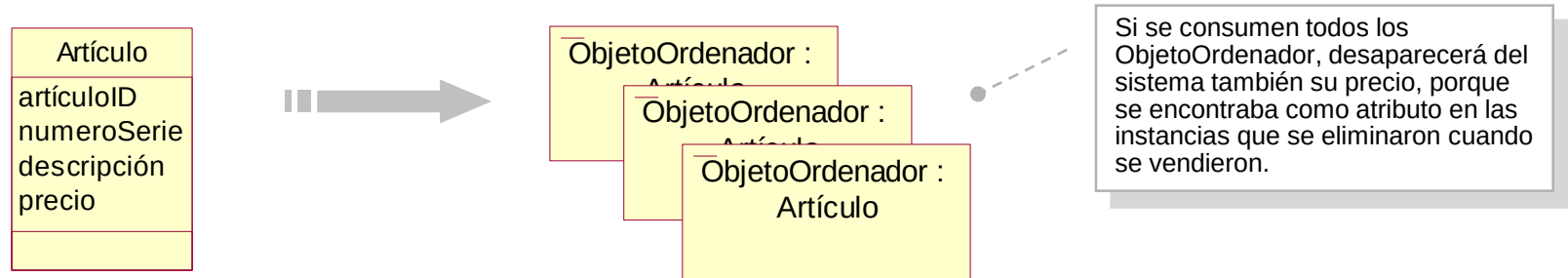
- informes: por lo general, no se recogen en el modelo de dominio porque muestran información derivada de otras fuentes, duplicando información.
 - a discutir: ¿es el **Recibo** una clase conceptual?

modelo del dominio: identificación de clases

- error habitual al identificar clases candidatas:
 - considerar como un atributo algo que debería ser un concepto
 - en caso de duda, debe considerarse un concepto separado, puesto que los atributos no deben ser muy habituales en un modelo del dominio

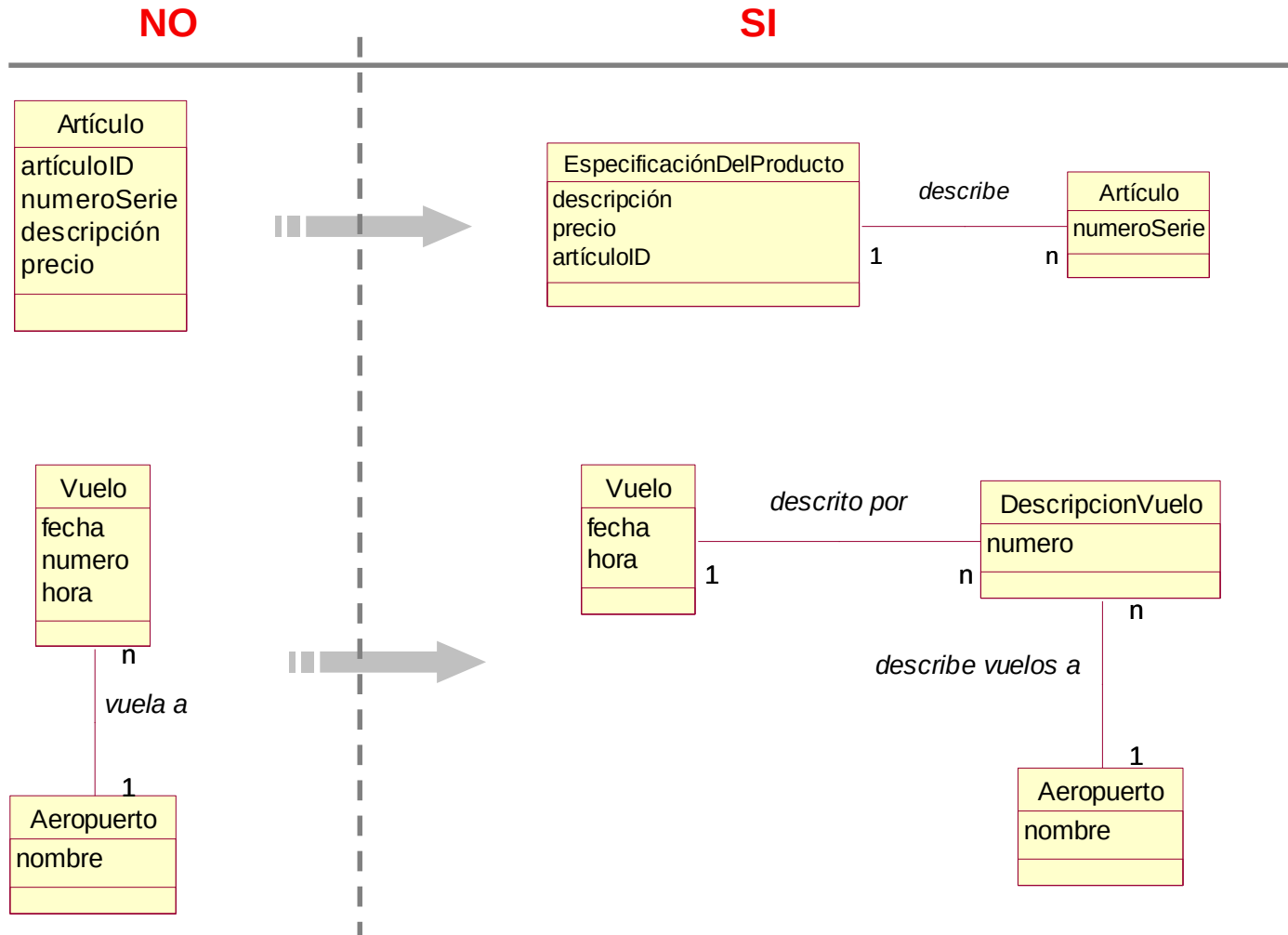


modelo del dominio: identificación de clases



- clases conceptuales de especificación o descripción
 - se utilizan para especificar o describir otras clases
 - una clase *EspecificaciónDelArtículo* no representa un *Artículo*, sino una descripción de la información **sobre** los artículos:
 - aunque se vendan todos los elementos inventariados, eliminándose las correspondientes instancias software de *Artículo*, permanecerá la *EspecificaciónDelArtículo*
 - habituales en entornos de gestión (sistemas de compras, ventas, inventarios,...) y fabricación (descripciones de los productos fabricados)
 - se usan cuando:
 - se necesita la descripción de un artículo o servicio independientemente de la existencia actual de algún ítem de esos artículos o servicios
 - la eliminación de instancias de las cosas que describen provocan pérdida de información que necesitaría mantenerse
 - reduce información redundante o duplicada

modelo del dominio: identificación de clases



modelo del dominio: identificación de clases

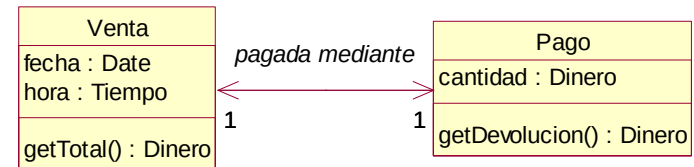
- reducción del salto en la representación (salto semántico):
 - una de las grandes ventajas de la tecnología de objetos
 - reducción de la diferencia entre el modo en el que el personal involucrado concibe el dominio y su representación en el software
 - ejemplo:
 - un pago en el Modelo del Dominio es una clase conceptual (un concepto)
 - un pago en el Modelo de Diseño es una clase software
 - no son lo mismo, pero el primero “inspiró” el nombre y definición del segundo

Modelo del Dominio

Vista del personal involucrado de los conceptos más relevantes del dominio
Visualiza los modelos del mundo real.



inspira objetos y
nombres en el
modelo de diseño



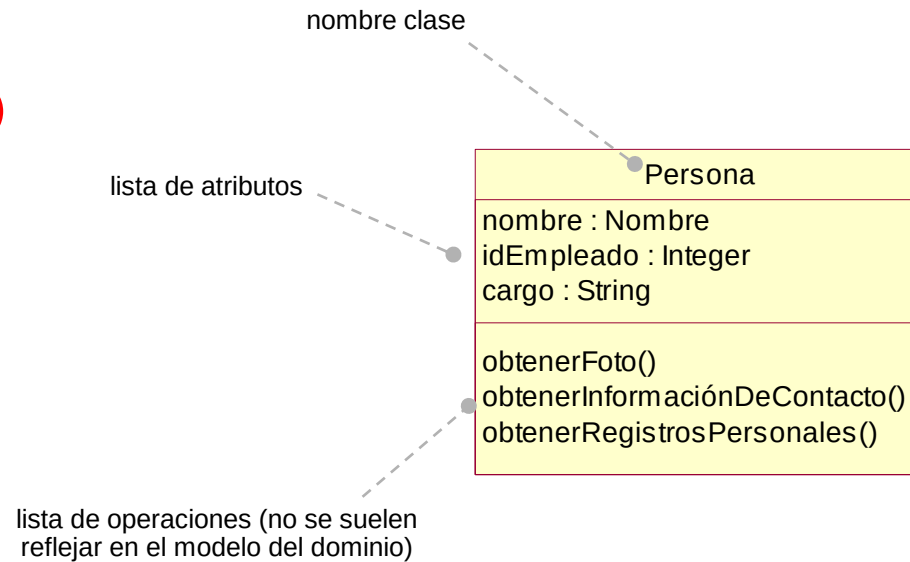
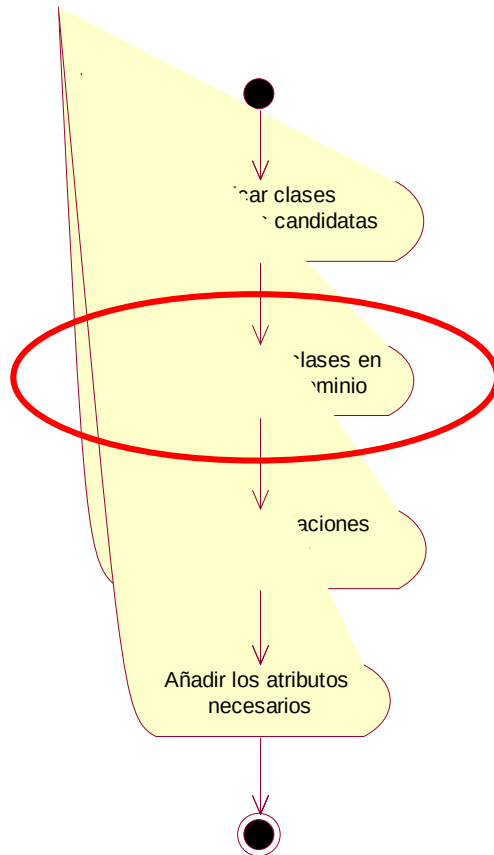
Modelo del Diseño

El desarrollador orientado a objetos se ha inspirado en el dominio del mundo real al crear las clases software.
Visualiza los componentes software

modelo del dominio: representar las clases

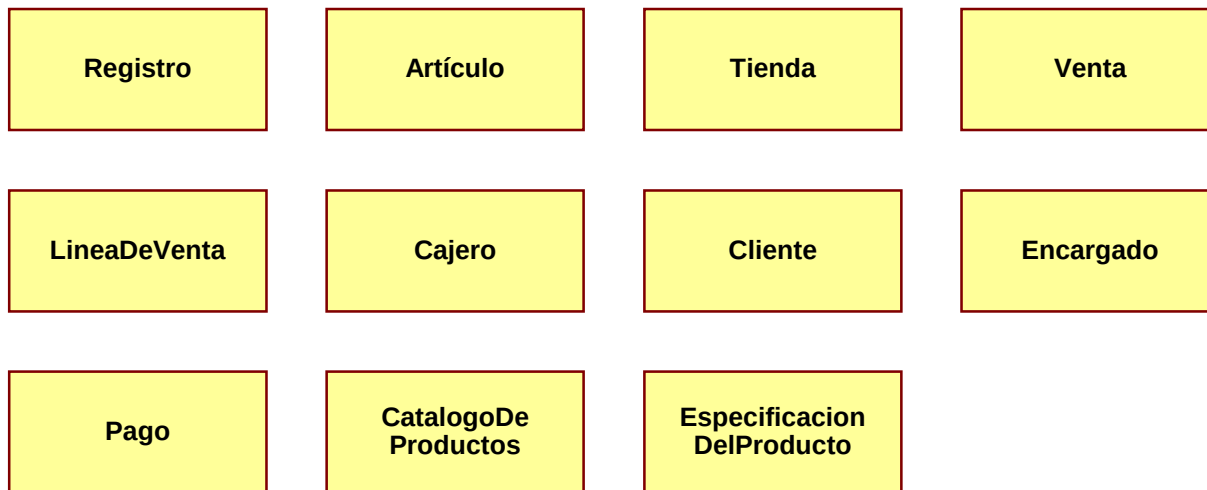
■ representar las clases en un modelo del dominio

■ se utiliza la notación de clase de UML

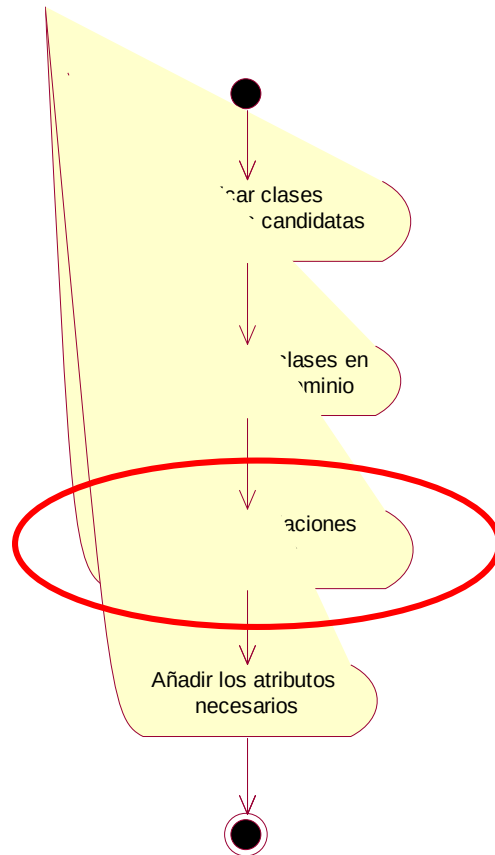


modelo del dominio: representar las clases

Representación de las clases conceptuales del Sistema PDV



modelo del dominio: asociaciones



■ asociación

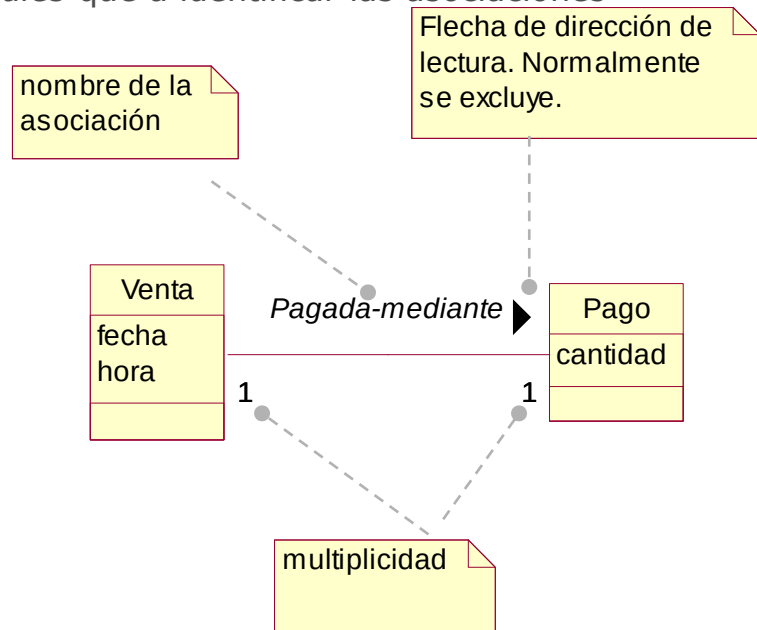
- según UML, una asociación es "la relación semántica entre dos o más clasificadores que implica conexiones entre sus instancias"

■ se registran las asociaciones:

- de las que es necesario conservar el conocimiento de la relación durante algún tiempo (por ejemplo, la relación entre *LíneaPedido* y *Pedido*)
- derivadas de la Lista de Asociaciones Comunes

■ importante: registrar únicamente asociaciones útiles para mantener el diagrama legible

■ es más importante dedicar tiempo a localizar las *clases conceptuales* que a identificar las asociaciones



modelo del dominio: asociaciones

- lista de asociaciones comunes
 - relación de categorías habituales que, normalmente, merece la pena tener en cuenta

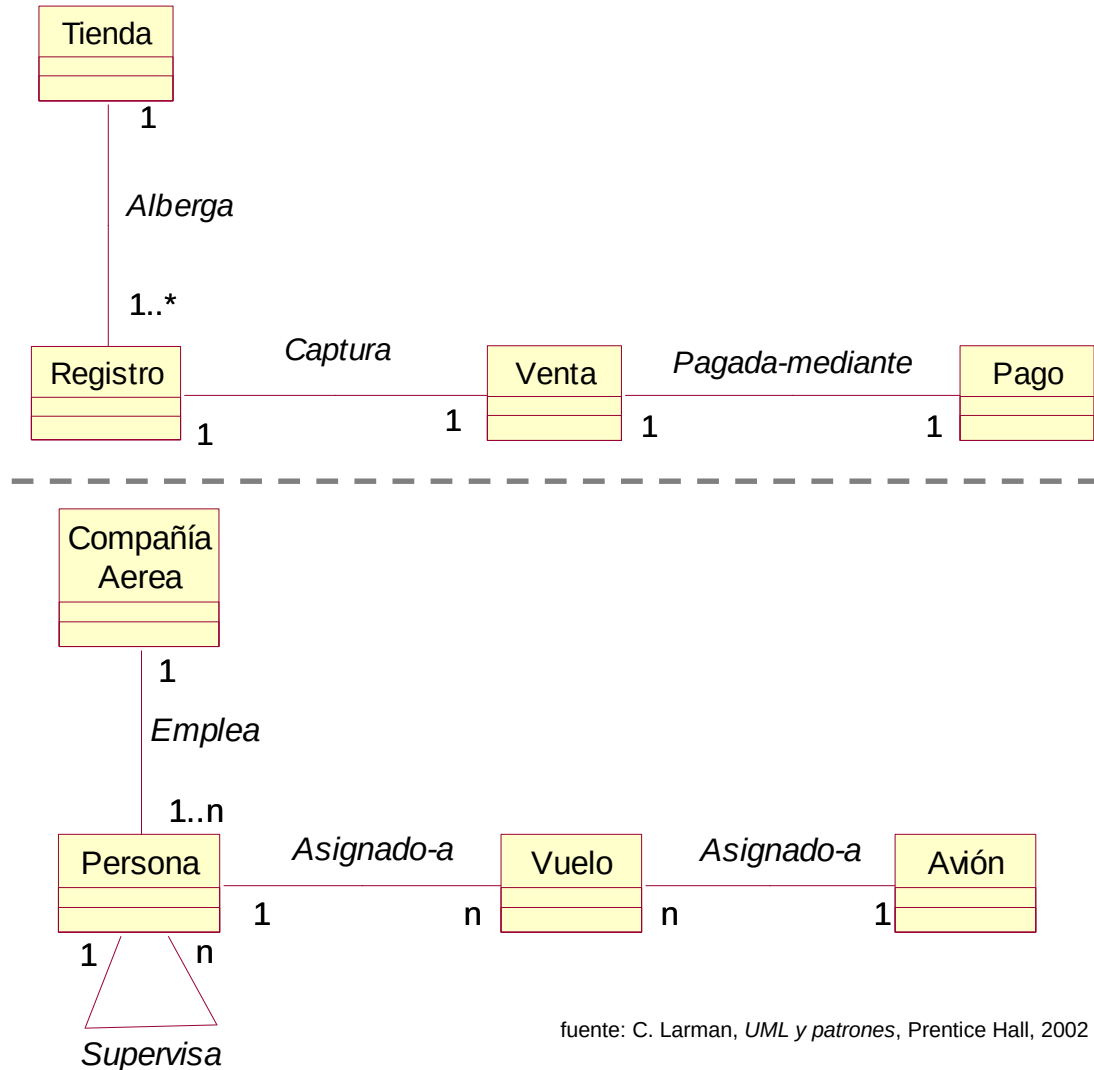
Categoría	Ejemplos
A es una parte física de B	Caja-Registro Ala-Avión
A es una parte lógica de B	LíneaDeVenta-Venta EtapasVuelo-RutaVuelo
A está contenido físicamente en B	Registro-Tienda, Artículo-Estantería Pasajero-Avión
A está contenido lógicamente en B	DescripciónArtículo-Catálogo Vuelo-PlanificaciónVuelo
A se conoce/registra/recoge/informa/captura en B	Venta-Registro Reserva-ListaPasajeros
A es una descripción de B	DescripciónDelArtículo-Artículo DescripciónDelVuelo-Vuelo
A es una línea de una transacción o importe de B	LíneaDeVenta-Venta TrabajoMantenimiento-RegistroDeMantenimiento
A es una subunidad organizativa de B	Departamento-Tienda Mantenimiento-CompañíaAerea

Categoría	Ejemplos
A es miembro de B	Cajero-Tienda Piloto-CompañíaAerea
A utiliza o gestiona B	Cajero-Registro Piloto-Avión
A se comunica con B	Cliente-Cajero AgenteReservas-Pasajero
A está relacionado con una transacción B	Cliente-Pago Pasajero-Billete
A es una transacción relacionada con otra transacción B	Pago-Venta Reserva-Cancelación
A está al lado de B	LíneaDeVenta-LíneaDeVenta Ciudad-Ciudad
A es propiedad de B	Registro-Tienda Avión-CompañíaAerea
A es un evento relacionado con B	Venta-Cliente, Venta-Tienda Salida-Vuelo

relaciones cuya inclusión en el Modelo de Dominio es especialmente útil

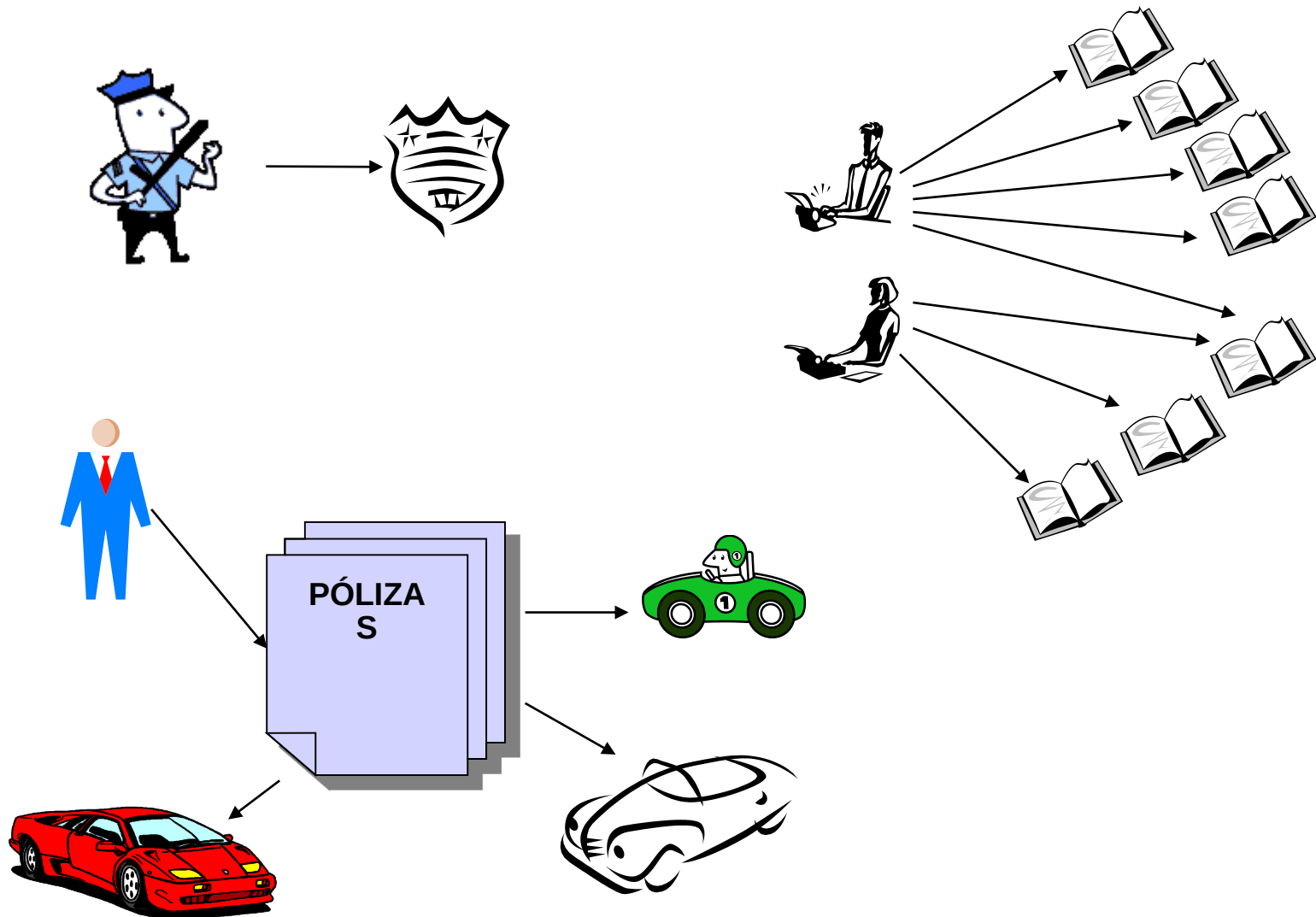
modelo del dominio: asociaciones

- nombre de las asociaciones
 - formato: *NombreTipo – FraseVerbal – NombreTipo* donde la frase verbal crea una secuencia legible y con significado en el contexto del modelo
 - normalmente la dirección por defecto para la lectura de los nombres de asociaciones es de izquierda a derecha o arriba abajo



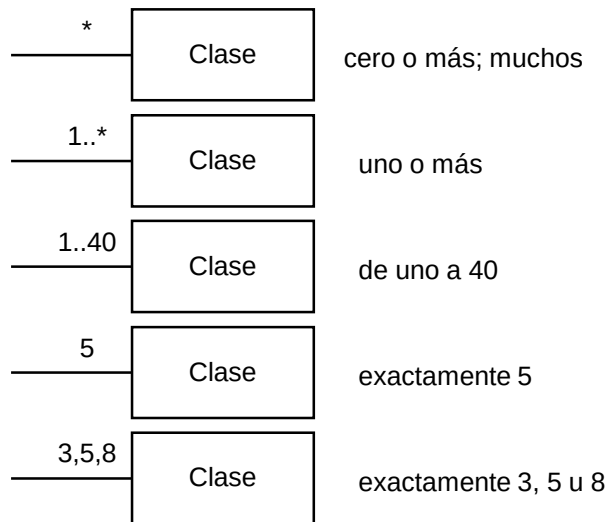
fuelle: C. Larman, *UML y patrones*, Prentice Hall, 2002

modelo del dominio: asociaciones



■ multiplicidad

- indica cuántas instancias de una clase A pueden asociarse con una instancia de una clase B
 - en un momento concreto
 - NO a lo largo de un periodo de tiempo
- indica una restricción de diseño que será (o podrá ser) reflejada en el software



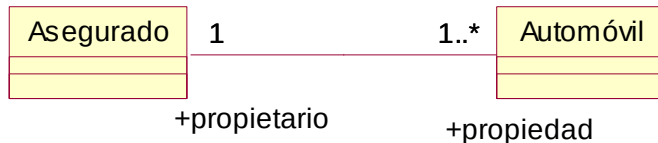
■ multiplicidad

- puede existir cualquier tipo de multiplicidad, pero en la práctica la mayoría de las asociaciones pertenecen a uno de los siguientes tipos

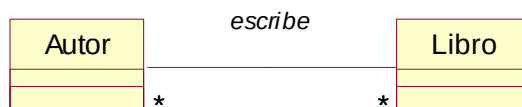
- asociación de uno a uno: tiene una multiplicidad de 1 a cada extremo. Significa que existe solamente un vínculo entre instancias de cada clase. Por ejemplo, *OficialPolicía* tiene exactamente un *NúmeroIdentificación*, y éste representa a uno y sólo a un *OficialPolicía*



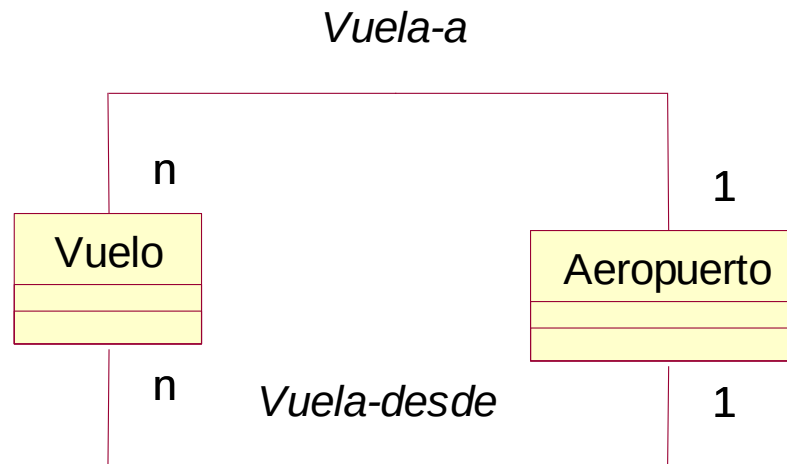
- asociación de uno a muchos: tiene una multiplicidad de 1 en un extremo y 0..n en el otro (a veces representado con un asterisco)



- asociación de muchos a muchos: tiene una multiplicidad de 0..n o 1..n en ambos extremos. Indica que pueden existir una cantidad arbitraria de vínculos entre instancias de las dos clases. Es el tipo más complejo de asociación.



- pueden existir dos o más asociaciones entre dos clases conceptuales

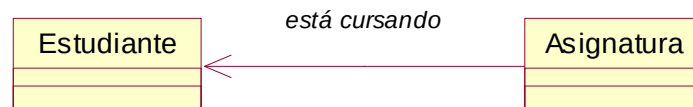


modelo del dominio: asociaciones

Categoría	Ejemplos
A es una parte física de B	<i>Registro-Caja</i>
A es una parte lógica de B	<i>LineaDeVenta-Venta</i>
A está contenido físicamente en B	<i>Registro-Tienda, Artículo-Tienda</i>
A está contenido lógicamente en B	<i>EspecificacionDelProducto-CatalogoDeProductos CatalogoDeProductos-Tienda</i>
A es una descripción de B	<i>EspecificacionDelProducto-Articulo</i>
A es una línea de una transacción o informe de B	<i>LíneaDeVenta-Venta</i>
A se conoce/registra/recoge/informa/captura en B	<i>(completa) Venta-Tienda (actual) Venta-Registro</i>
A es miembro de B	<i>Cajero-Tienda</i>
A es una subunidad organizativa de B	<i>No aplicable</i>
A utiliza o gestiona B	<i>Cajero-Registro Encargado-Registro Encargado-Cajero, aunque probablemente no es aplicable</i>
A se comunica con B	<i>Cliente-Cajero</i>
A está relacionado con una transacción B	<i>Cliente-Pago Cajero-Pago</i>
A es una transacción relacionada con otra transacción B	<i>Pago-Venta</i>
A está al lado de B	<i>LineaDeVenta-LineaDeVenta</i>
A es propiedad de B o informe de B	<i>Registro-Tienda</i>

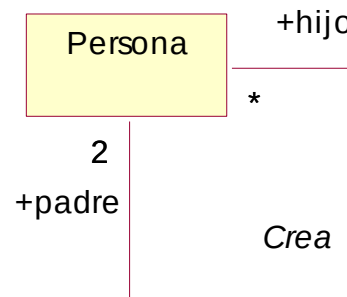
■ navegabilidad

- indica que es posible enviar mensajes en la dirección de la flecha en uno de los dos extremos de asociación
- no hay que especificarla hasta que el diagrama de clases sea estable



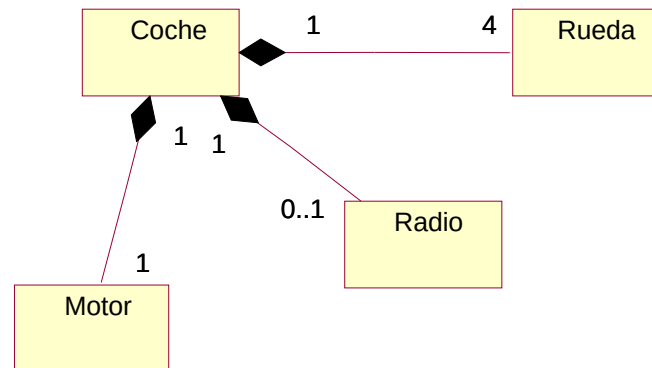
En este caso, la clase Asignatura *conoce* a la clase Estudiante, pero no al revés.

■ asociaciones reflexivas



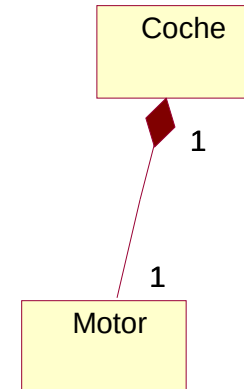
■ agregación

- es un tipo de asociación que se utiliza para modelar las relaciones *todo-parte* entre las cosas. El todo se denomina **compuesto**
- normalmente se identifica en el Modelo de Diseño, pero puede ser útil o necesario identificar algunas relaciones de agregación en el Modelo del Dominio.
- representación en UML: un rombo hueco o relleno en el extremo del compuesto de una asociación todo-parte
- el nombre de la asociación se suele excluir porque se piensa normalmente como *Tiene-parte*



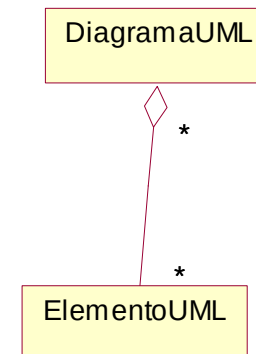
■ agregación de composición

- la parte es un miembro de un único objeto compuesto y existe una dependencia de existencia y disposición de la parte sobre el compuesto
- la multiplicidad en la parte del compuesto sólo puede ser 1
- ejemplo: en una relación *Coche* – *Motor*, el motor tiene sentido como tal (existe) únicamente si está asociado a un coche
- representación en UML: un rombo relleno



■ agregación compartida

- la multiplicidad en el extremo del compuesto puede ser más de uno: la parte puede estar simultáneamente en muchas instancias del compuesto
- se utiliza para conceptos abstractos, no físicos
- representación en UML: un rombo hueco



- identificación de la agregación
 - si hay duda, se descarta
 - se debe mostrar una agregación:
 - cuando el tiempo de vida de la parte está ligado al tiempo de vida del compuesto (existe una dependencia de creación – eliminación de la parte en el todo).
 - cuando existe un ensamblaje obvio todo-parte físico o lógico
 - cuando alguna propiedad del compuesto se propaga a las partes, como la ubicación
 - cuando las operaciones que se aplican sobre el compuesto se propagan a las partes, como la destrucción, movimiento o grabación



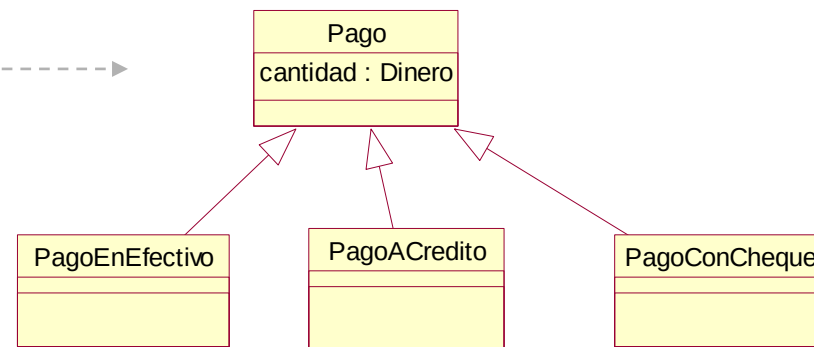
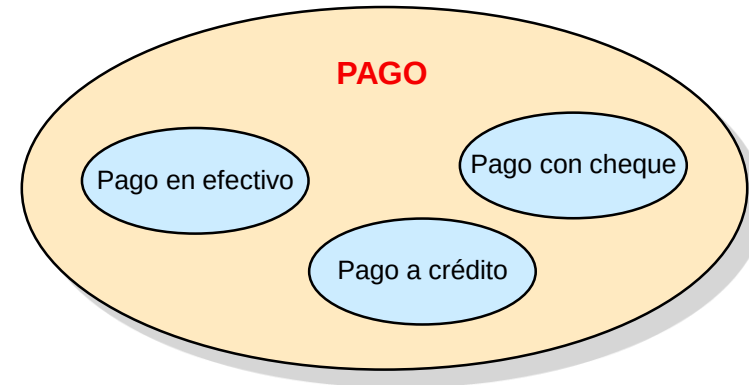
■ generalización

- actividad de identificar elementos comunes entre los conceptos y definir las relaciones de superclase (concepto general) y subclase (concepto especializado). Así, los conceptos se representan en jerarquías de clases.
- **superclase conceptual:** su definición es más general y abarca más que la definición de una subclase
- **subclase conceptual:** debe ser un miembro del conjunto de la superclase, es decir, *es un tipo de* superclase
- todos los miembros del conjunto de una subclase conceptual son miembros del conjunto de su superclase

Un *Pago* representa la transacción de transferir dinero para una compra de una parte a otra.

PagoACredito es una transferencia de dinero por medio de una institución de crédito que autoriza la operación.

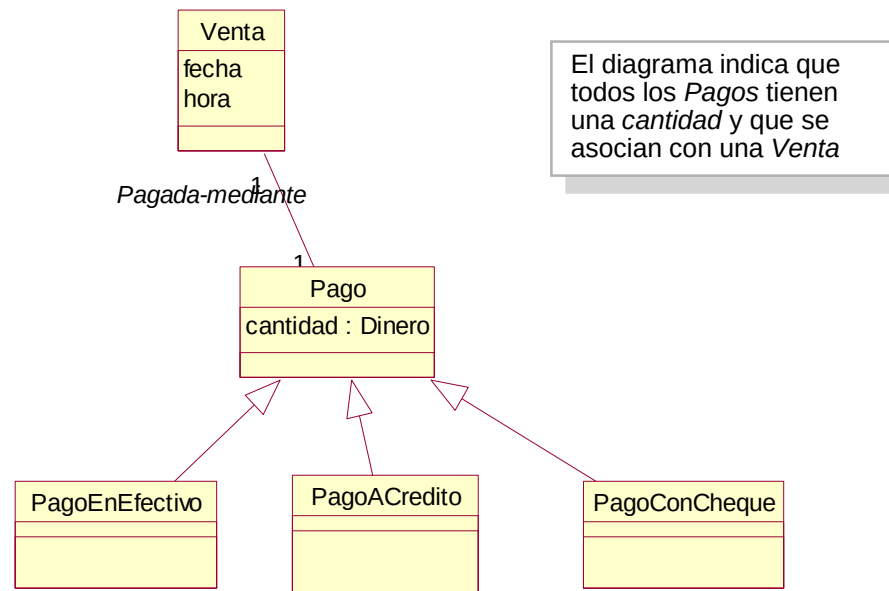
Por tanto, *Pago* es una definición más amplia y general que la de *PagoACredito*.



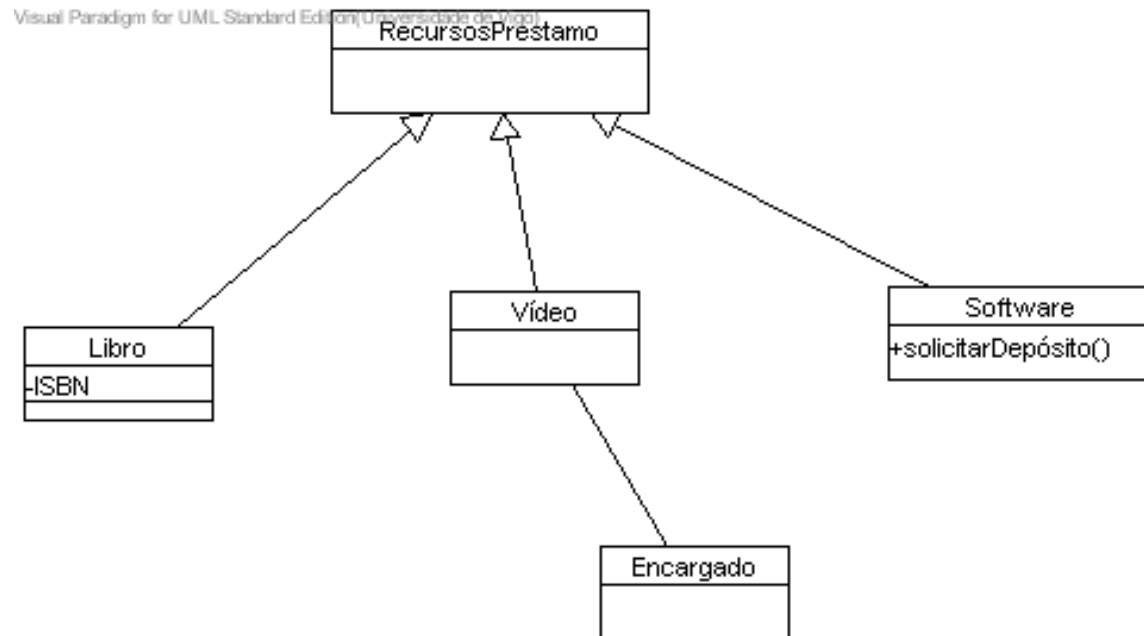
fuelle: C. Larman, *UML y patrones*, Prentice Hall, 2002

■ jerarquía de clases:

- las declaraciones sobre las superclases se aplican a las subclases
- regla del 100%: el 100% de la definición de la superclase conceptual se debe de poder aplicar a la subclase. La subclase debe ajustarse al 100% de los atributos y asociaciones de la superclase



fuelle: C. Larman, *UML y patrones*, Prentice Hall, 2002



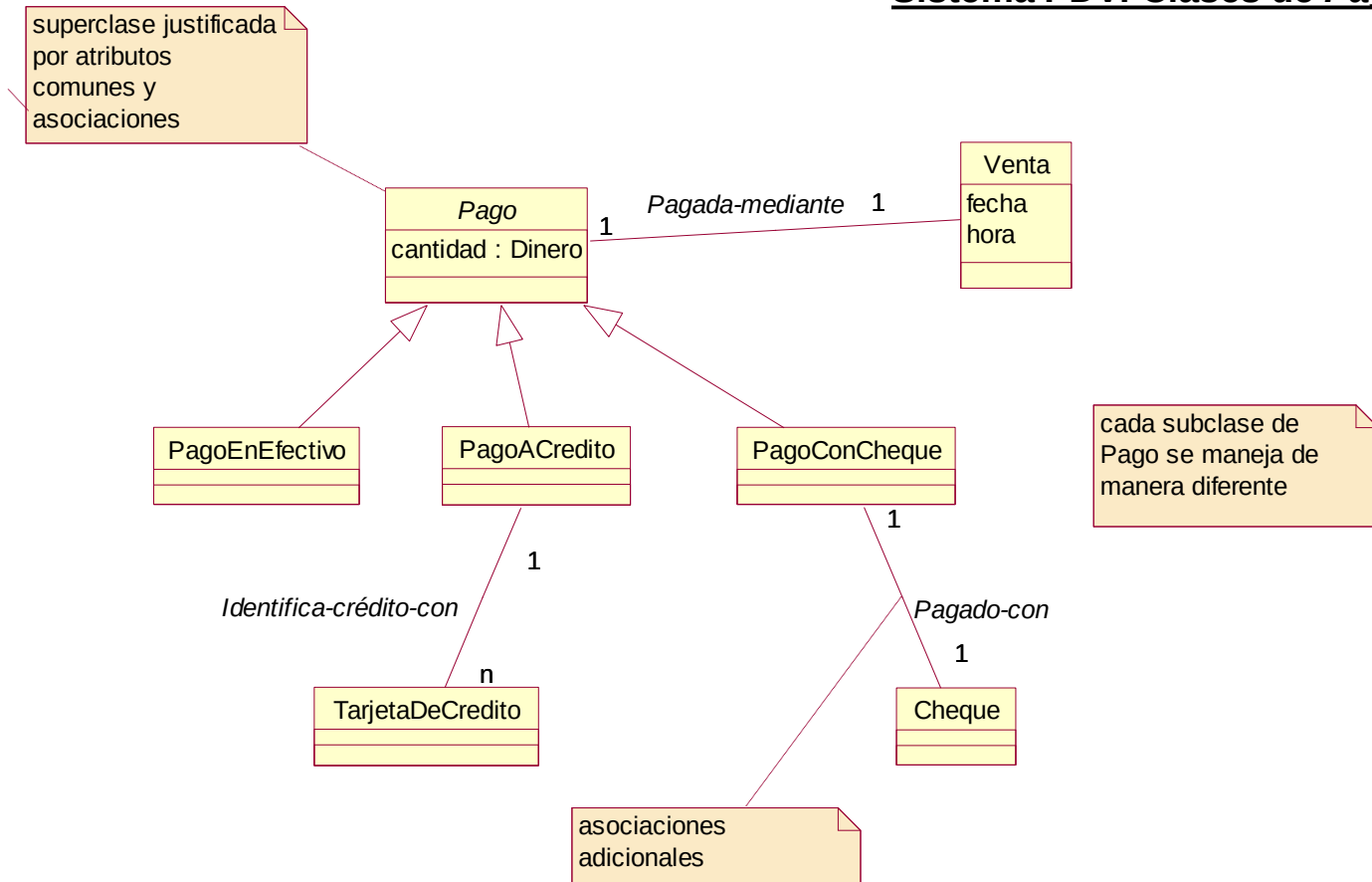
- cuando dividir una clase conceptual en subclases
 - cuando la subclase tiene atributos adicionales de interés
 - cuando la subclase tiene asociaciones adicionales de interés
 - cuando el concepto de la subclase funciona, se maneja, reacciona o se manipula de manera diferente a la superclase o a otras subclases, de alguna manera que es interesante.
 - cuando el concepto de la subclase representa una cosa animada (animal, maquinaria,...) que se comporta de manera diferente a la superclase o a otras subclases, de alguna manera que resulta interesante poner de manifiesto en el modelo del dominio.

Motivación de la subclase conceptual	Ejemplos
La subclase tiene atributos adicionales de interés	Pagos: no aplicable Biblioteca: <i>Libro</i> , subclase de <i>RecursoPrestable</i> , tiene un atributo ISBN
La subclase tiene asociaciones adicionales de interés	Pagos: <i>PagoACredito</i> , subclase de <i>Pago</i> , asociada con una <i>TarjetaDeCredito</i> Biblioteca: <i>Vídeo</i> , subclase de <i>RecursoPrestable</i> , asociada con un <i>Encargado</i>
El concepto de la subclase funciona, se maneja, reacciona o se manipula de manera diferente a la superclase o a otras subclases, de alguna manera que es interesante	Pagos: <i>PagoACredito</i> , subclase de <i>Pago</i> , se gestiona de manera diferente a otros tipos de pagos en el modo de autorizarlo Biblioteca: <i>Software</i> , subclase de <i>RecursoPrestable</i> , requiere un depósito antes de que pueda prestarse
El concepto de la subclase representa una cosa animada (personal, maquinaria, animal...) que se comporta de manera diferente a la superclase o a otras subclases, de alguna manera que resulta interesante poner de manifiesto en el modelo del dominio	Pagos: no aplicable Biblioteca: no aplicable Investigación de Mercado: <i>Hombre</i> , subclase de <i>Persona</i> , se comporta de manera diferente a la <i>Mujer</i> con respecto a los hábitos de compras

- cuando definir una superclase conceptual
 - cuando las subclases potenciales representan variaciones de un concepto similar
 - cuando las subclases se ajustarán a las reglas del 100% y la regla *Es-Un-Tipo-De*
 - cuando todas las subclases tienen un mismo atributo que se puede expresar en la superclase
 - cuando todas las subclases tienen la misma asociación que se puede unir y relacionar con la superclase
- jerarquías de clases y herencia en el software
 - la **herencia**
 - es un mecanismo software para hacer que las características de la superclase se apliquen a las subclases
 - es un concepto que no se utiliza en el Modelo del Dominio, pero sí cuando se pasa al Modelo de Diseño y Modelo de Implementación
 - las jerarquías de clases conceptuales del Modelo del Dominio pueden reflejarse o no en el Modelo de Diseño, dependiendo de las características del lenguaje y otras cuestiones

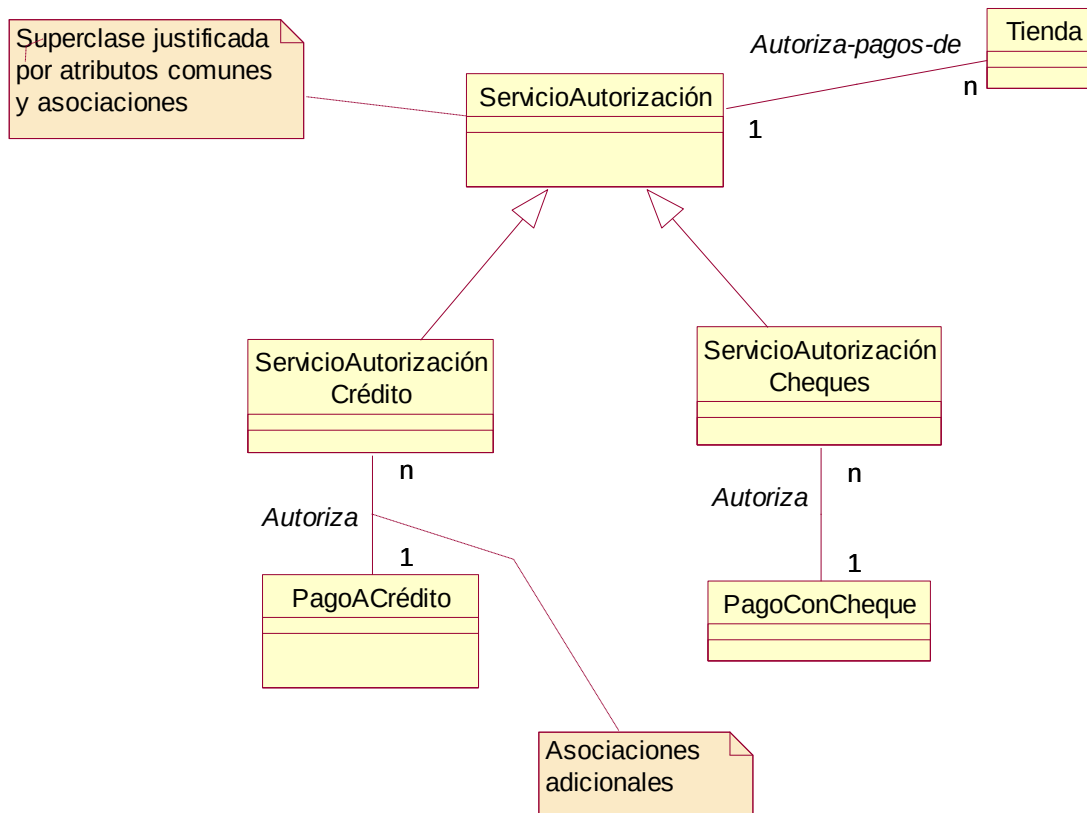
modelo del dominio: asociaciones

Sistema PDV: Clases de Pago



fuelle: C. Larman, *UML y patrones*, Prentice Hall, 2002

Sistema PDV: Clases de *ServicioAutorización*



fuelle: C. Larman, *UML y patrones*, Prentice Hall, 2002

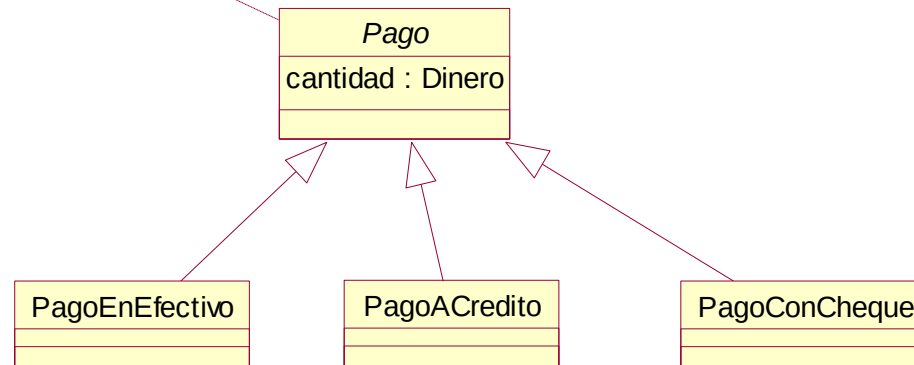
■ clases conceptuales abstractas

- útil identificarlas pues se restringe las clases que pueden tener instancias concretas y por tanto se clarifican las reglas del dominio del problema

■ regla general:

- si cada miembro de una clase C debe ser también un miembro de una subclase, entonces la clase C se denomina **clase conceptual abstracta**.

la clase abstracta se representa en cursiva



clases asociación

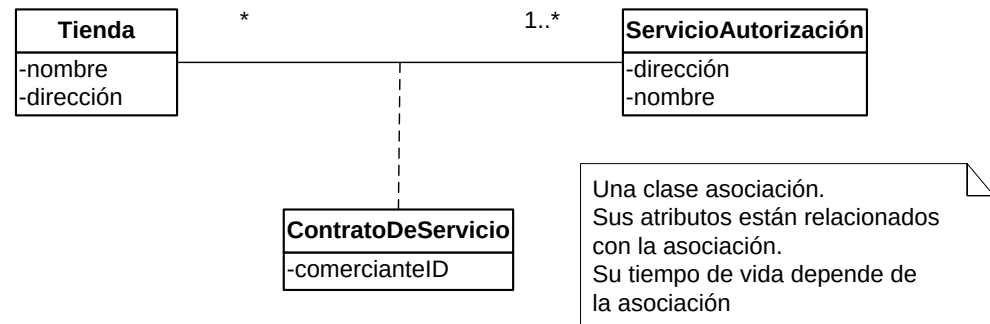
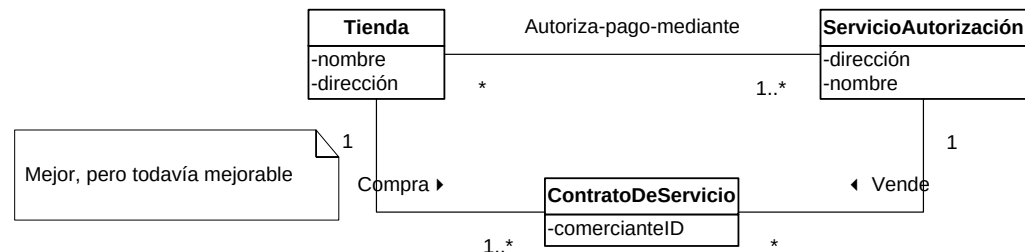
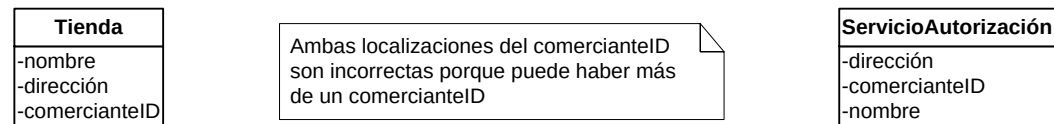
- en un modelo del dominio, si una clase C puede tener simultáneamente muchos valores para el mismo tipo de atributo A, no se colocará este atributo en C, sino en otra clase asociada con C.

■ ejemplo:

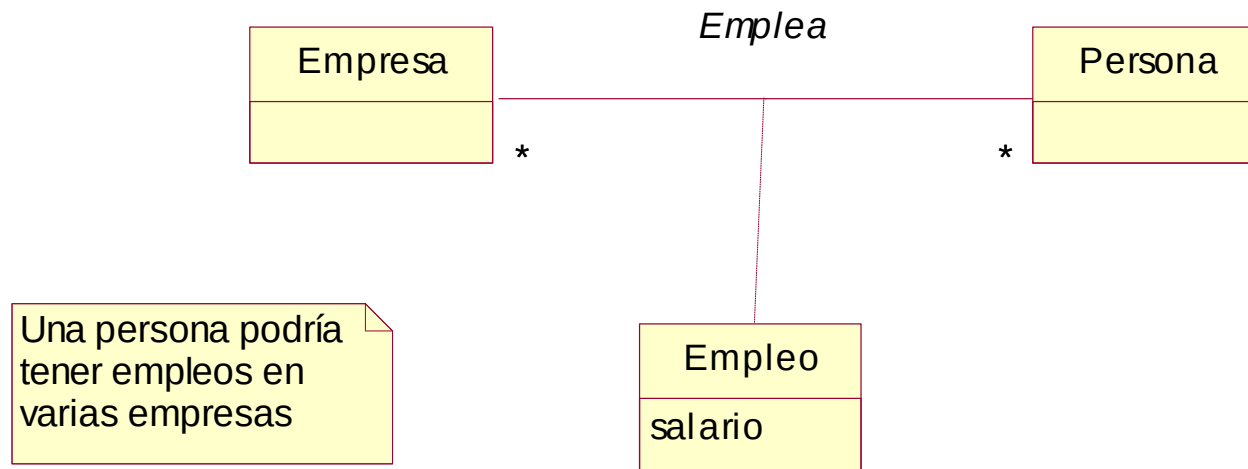
- una *Persona* puede tener muchos números de teléfono. Se colocará el número de teléfono en otra clase, como *NúmeroTeléfono* o *InformaciónDeContacto*, y se asocian muchas de estas clases a *Persona*.

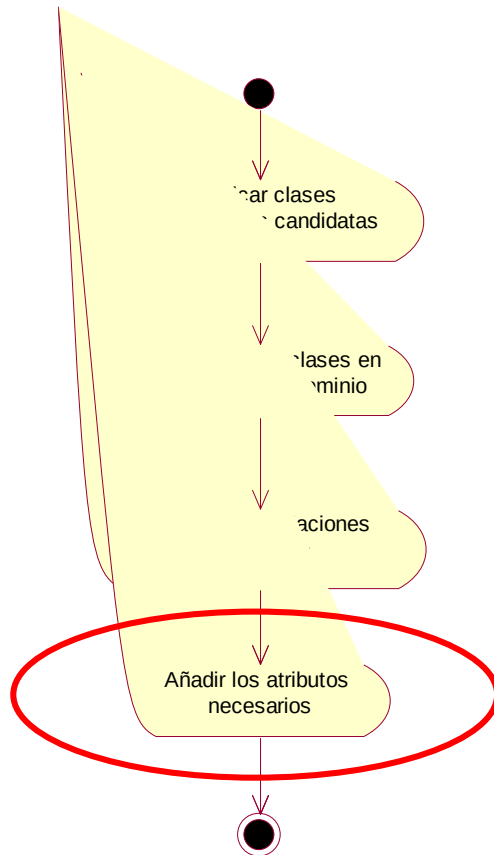
■ guía

- hay un atributo relacionado con una asociación
- el tiempo de vida de las instancias de la clase asociación depende de la asociación
- existe una asociación muchos-a-muchos entre dos conceptos, e información asociada con la propia asociación



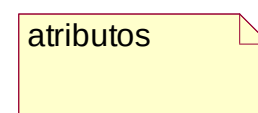
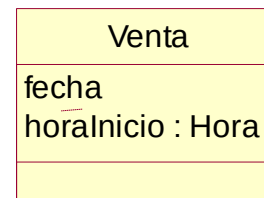
fuelle: C. Larman, *UML y patrones*, Prentice Hall, 2002





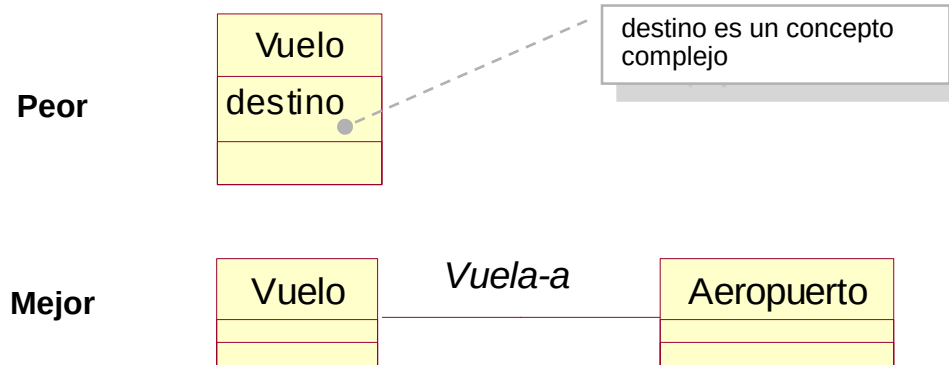
■ atributo

- es un valor de datos lógico de un objeto
- se incluyen aquellos atributos para los que los requisitos indican la necesidad de registrar su información
- ejemplo: un recibo recoge la información de una venta, incluyendo fecha y hora. La dirección quiere conocer fecha y hora de las ventas. Por tanto, la clase *Venta* necesita los atributos *fecha* y *hora*
- notación UML: se muestran en el segundo compartimento del rectángulo de la clase. Los tipos se muestran opcionalmente



■ tipos de atributos

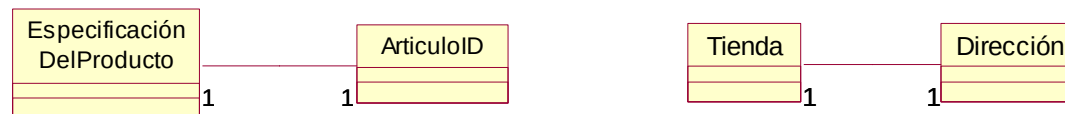
- en el modelo de dominio deben usarse preferiblemente
 - atributos simples: no deben ser conceptos de dominio complejos (*Venta, Aeropuerto, ...*)
 - tipos de datos:
 - ◆ principalmente booleano, fecha, número, string y hora
 - ◆ otros: Dirección, Color, Teléfono, NIF, Código Postal,...
- importante: relacionar las clases conceptuales con una asociación, no con un atributo. En caso de duda, se debe optar por usar una clase conceptual antes que un atributo.
- durante el diseño y la implementación podrán aparecer nuevos atributos que referencian a otros objetos software complejos, pero que no tienen sentido en el Modelo de Dominio



■ tipos de datos como clases

- un atributo que puede considerarse como un tipo de dato primitivo puede representarse como una clase si:
 - está compuesto de secciones separadas (número de teléfono, nombre de persona,...)
 - hay operaciones asociadas con él, como algoritmos de validación (NIF, número de la Seguridad Social,...)
 - tiene otros atributos (el precio de una oferta puede tener una fecha de comienzo y otra de fin)
 - es una cantidad con una unidad de medida (la cantidad de un pago tiene una unidad monetaria)
 - es una abstracción de uno o más tipos con estas cualidades (un identificador de artículo en el dominio de ventas es una generalización de tipos como el *Código de Producto Universal -UPC-* o el *Número de Artículo Europeo -EAN-*)
- ejemplo: en el sistema de Punto de Venta las clases *ArticuloID*, *Dirección* y *Cantidad* son tipos de datos pero se pueden considerar como clases independiente debido a sus características
- representación: como clase independiente o en el compartimento de atributos de la clase, dependiendo de la utilización del modelo del dominio y la importancia de los conceptos en el dominio

Correcto



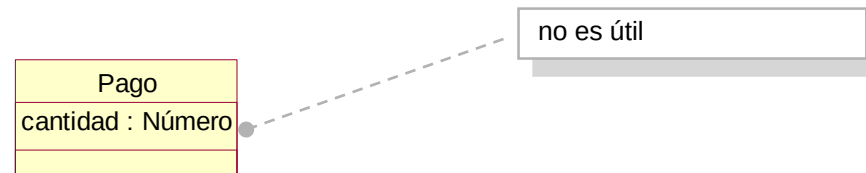
Correcto



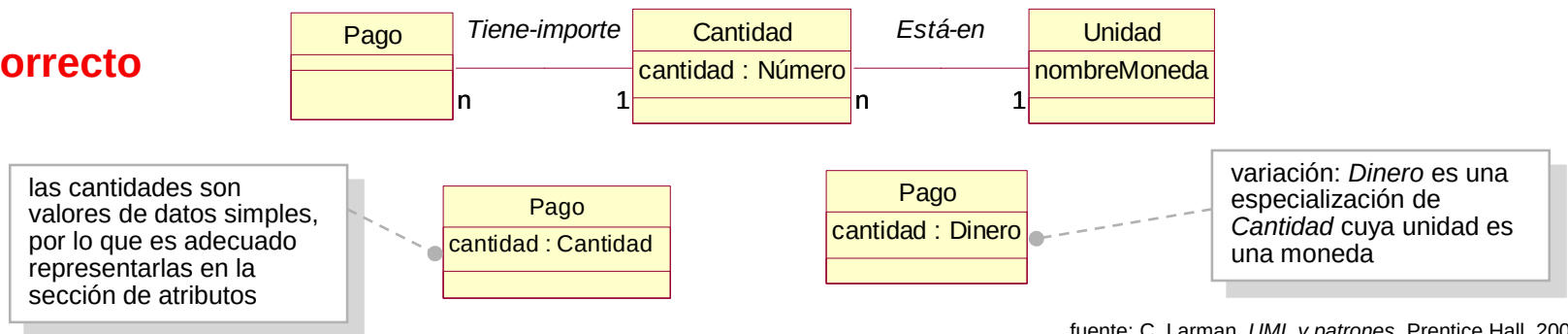
fuelle: C. Larman, *UML y patrones*, Prentice Hall, 2002

- cantidades y unidades de los atributos
 - la mayoría de las cantidades numéricas llevan unidades asociadas no deben representarse únicamente como números
 - ejemplos: precio, velocidad, peso ...
 - solución: representar la *cantidad* como una clase conceptual aparte con una *unidad* asociada

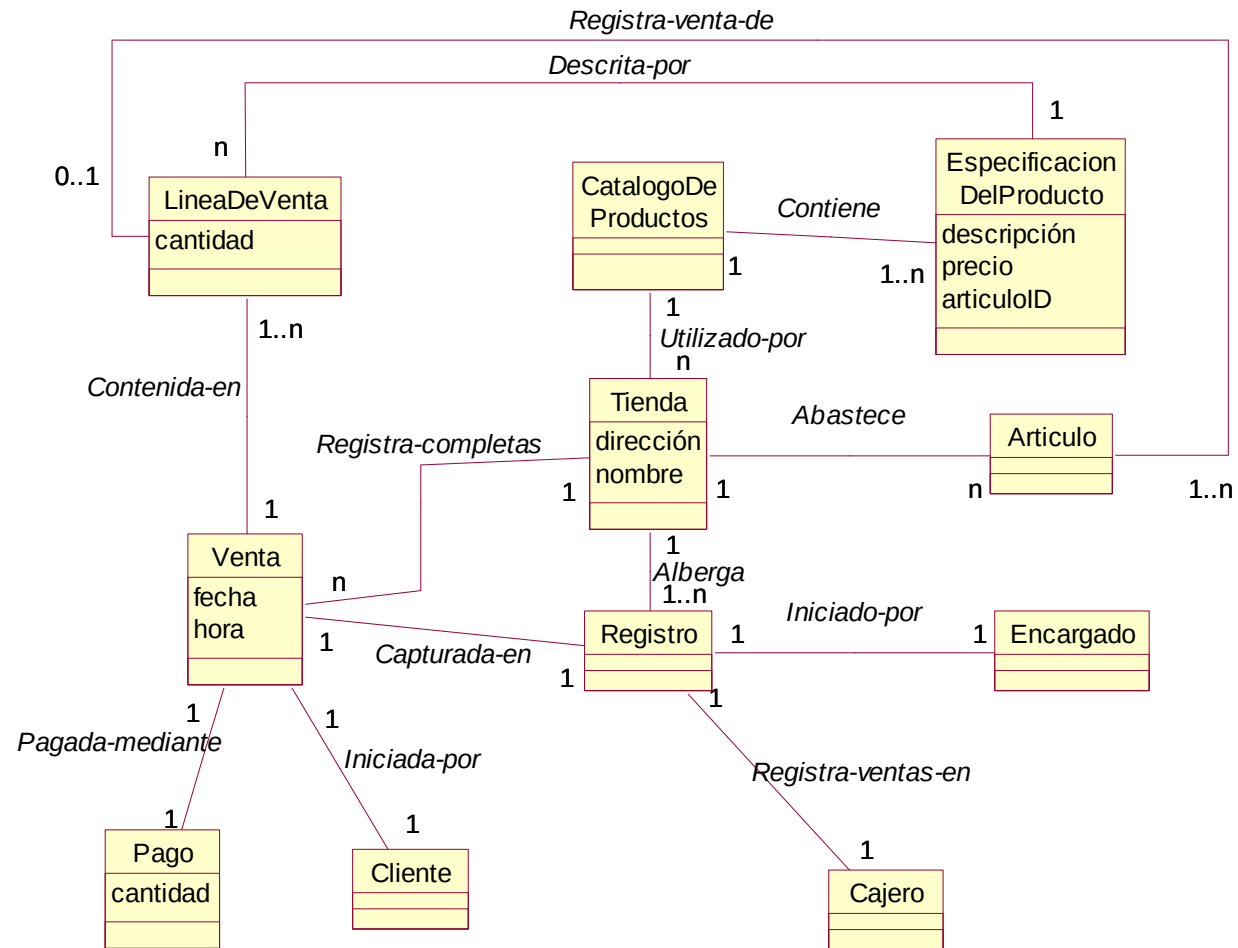
Incorrecto



Correcto



fuelle: C. Larman, *UML y patrones*, Prentice Hall, 2002

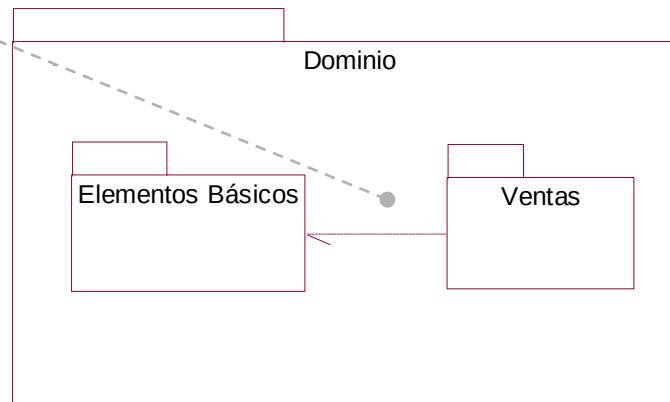


fuelle: C. Larman, *UML y patrones*, Prentice Hall, 2002

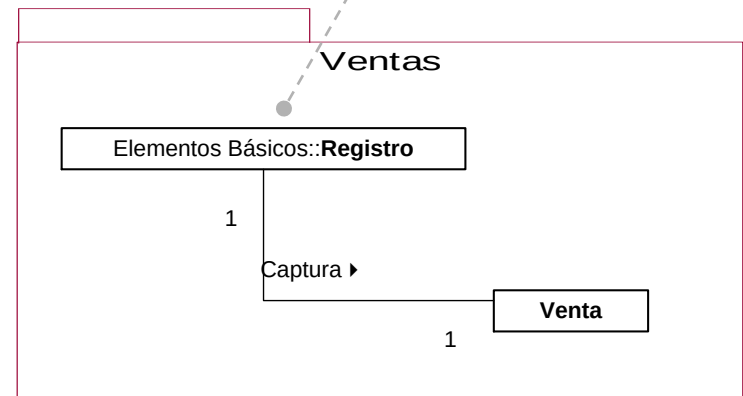
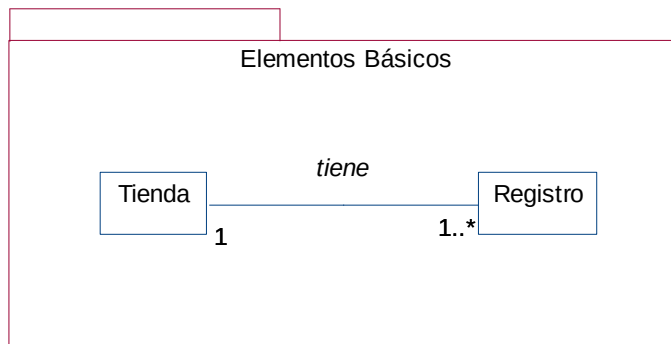
- los modelos de dominio se pueden dividir en paquetes cuando han crecido demasiado
 - los paquetes incluyen conceptos fuertemente relacionados
 - mejora la comprensión
 - permite realizar tareas de análisis en paralelo, de tal forma que diferentes equipos o personas analizan diferentes subdominios
- notación de paquetes en UML
 - paquete: se representa por una carpeta
 - pueden mostrarse dentro de un paquete otros paquetes subordinados
 - un elemento *pertenece* al paquete donde está definido, pero puede ser referenciado en otros paquetes, utilizando el formato *NombrePaquete::NombreElemento*

modelo del dominio: utilización de paquetes

Relación de dependencia:
indica que los elementos
del paquete dependiente
(Ventas) conocen o están
acoplados de algún modo
con los elementos del
paquete destino
(Elementos Básicos).

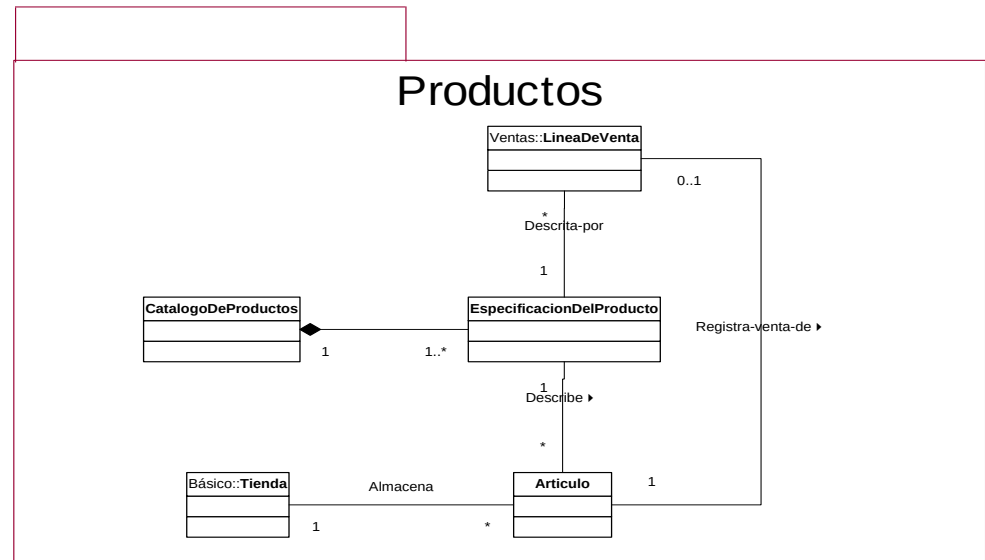
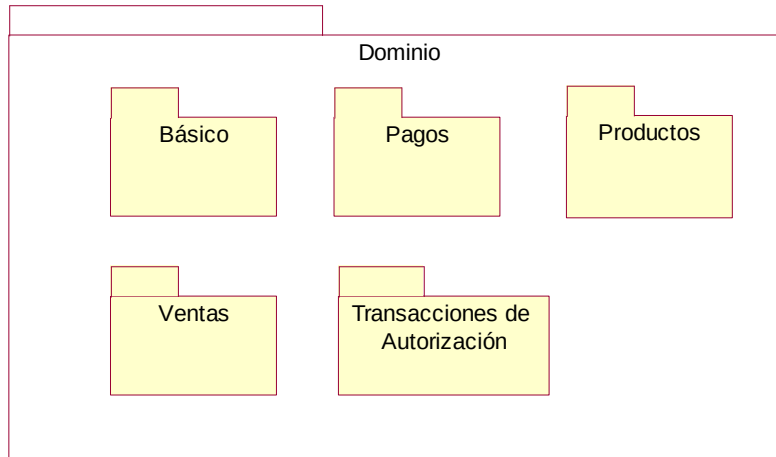


Una clase referenciada
en un paquete



fuelle: C. Larman, *UML y patrones*, Prentice Hall, 2002

modelo del dominio: utilización de paquetes



fuelle: C. Larman, *UML y patrones*, Prentice Hall, 2002

- cómo dividir el Modelo del Dominio
 - se deben poner juntos en paquetes los elementos que:
 - se encuentran en el mismo área de interés (están estrechamente relacionados por conceptos u objetivos)
 - están juntos en una jerarquía de clases
 - participan en los mismos casos de uso
 - están fuertemente asociados
 - consejo
 - utilizar un paquete *Dominio* que será el raíz de todos los elementos relacionados con el modelo del dominio
 - utilizar un paquete *Elementos Básicos*, o *Conceptos Comunes*, para englobar todos los elementos compartidos, comunes a otros elementos, básicos,...

Larman, C., *UML y patrones*, cap. 10, 11, 12, 13, 26 y 27

Jacobson, I., Booch, G. y Rumbaugh, J., *El Proceso Unificado de Desarrollo*, cap. 8

Stevens, P., *Utilización de UML en ingeniería del software con objetos y componentes*, cap. 5, 6, 9, 10, 11 y 12

Bruegge, B., Dutoit, A.H., *Ingeniería del Software Orientado a Objetos*, cap. 5
